

# Advanced ACL Exploration

There's more to Access Control Lists than just allowing IP traffic into or out from an interface. In this CCNA Prep session, you'll learn the power of 5 tuple matching. Not sure what a 'tuple' is? No problem, we'll explain it to you. You'll also learn where ACLs show up beyond interface access-groups. From device management, NAT, NTP and quality services, ACLs play a pivotal role in an optimized and secure network. Get ready to crush any ACL question on your CCNA exam by joining in.

## 5.6 Configure and verify access control lists

In this lab,

## Setup and Scenario

This set of lab based demonstrations include a small network made up of "Clients", "Services", and "Simulated Internet" network locations connected by a router. There are 2 client hosts, an internal web server hosting 2 web sites, and a simulated internet with 3 web sites.

The internal web sites are:

- `http://server` - A network management application
- `http://adminserver:8080` - A system dashboard and management application (\*Note: This site is available on port 8080.)

The simulated internet web sites are:

- `http://internet100` - A streaming site
- `http://internet200` - An online game
- `http://internet210` - Landing page for a new AAI company

Each lab exercise will have you leverage access control lists, primarily extended access control lists, to enforce some policy.

Note: The credentials for all devices and hosts are `cisco / cisco`.

Be sure to **START** the lab before continuing to the demo labs.

## Pre-validation

The start state of the lab has the router and all hosts configured for IP connectivity end to end. Before we begin controlling access using ACLs, verify that traffic is flowing everywhere without problems.

## Client to Web Site Testing

Both clients should be able to communicate with all 5 sites listed above. Open a VNC connection from each `client`, launch firefox from the "Applications > Internet" menu, and navigate to each of the 5 sites listed above.

Note: Don't use the "Web Browser" option from the menu or the launchbar.

If the VNC desktop view isn't performing well, you can also use a console connection and `curl -v -o /dev/null {web_address}` to verify connectivity. For example:

```
client1:~$ curl -v -o /dev/null http://server

* Host server:80 was resolved.
* IPv6: (none)
* IPv4: 172.16.0.65
  % Total    % Received % Xferd  Average Speed   Time    Time       Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0*   Trying 172.16.0.65:80...
* Connected to server (172.16.0.65) port 80
* using HTTP/1.x
> GET / HTTP/1.1
> Host: server
> User-Agent: curl/8.14.1
> Accept: */*
>
< HTTP/1.1 200 OK                <----- Look for the 200 OK
< Server: nginx/1.27.5
< Date: Mon, 17 Nov 2025 03:06:20 GMT
< Content-Type: text/html
< Content-Length: 4688
< Last-Modified: Mon, 17 Nov 2025 02:50:55 GMT
< Connection: keep-alive
< ETag: "691a8d8f-1250"
< Accept-Ranges: bytes
<
{ [4688 bytes data]
100 4688 100 4688    0     0  98k      0  --:--:-- --:--:-- --:--:-- 106k
* Connection #0 to host server left intact
```

## Client to Router Testing

The `router` has several device services enabled that should be reachable from both `clients`.

1. Verify SSH access to the router from both clients.

```
ssh cisco@192.168.0.1
```

2. Verify SNMP access to the router from both clients.

```
snmpwalk -v 2c -c CCNAPREP 192.168.0.1 1.3.6.1.2.1.1.5.0
```

Note: You should get an output with the hostname (ie `sysName`) returned

3. Verify NTP access to the router from both clients.

```
ntpd -dw -p 192.168.0.1
```

Note: You will likely get a reply such as `ntpd: reply from 192.168.0.1: peer is unsynced`. This is expected and indicates that NTP communication is working, but the `router` isn't sync'd to a time source. All we care about is that NTP communication is working.

## Lab 1: Manage Client Access to Internal Web Servers

For this first lab, we will use an extended ACL to enforce the policy that **all clients** should be able to reach the internal web site `http://server`, but **only client1** (from the primary IP address of `192.168.0.17`) should be able to reach the internal web site `http://adminserver:8080`. Also, we should explicitly default to **permitting** traffic not explicitly targeted by this policy.

Let's break the goal down into individual details so we can build and apply the ACL.

- We are using an extended ACL, so we'll follow the rule of thumb to apply it close to the source. That would be interface `e0/1 inbound`
- The two web sites are hosted on the same server and IP address, but on different ports. This means matching on destination host IP address will NOT be enough. We'll need to use protocol and port information
- The *implicit deny* at the end of the ACL will need to be blocked by an *explicit permit*
- While we COULD rely on the *explicit permit* to allow all clients to reach `http://server`, as it is a specific policy, we should have an ACE (access control entry) explicitly match this traffic.

### What exactly does "5 Tuple" mean?

You may have heard the phrase "5 Tuple" when learning about extended ACLs... what does it actually mean? And what is a "tuple"?

A "tuple" is a computer engineering term that simply means a collection of data points of known meaning. Consider the  $(x, y)$  grid coordinance on a graph. That would be a tuple with 2 bits of data. Or maybe we're working in three demensions and it's  $(x, y, z)$ . Now it's a tuple with 3 bits of data.

So a "5 tuple" is simply a set of 5 pieces of data. For an extended access control list, the pieces of data relate to the layer 3 and 4 source and destination information for network traffic. The 5 tuples are:

- Protocol: The common options are `ip`, `tcp`, and `udp`. But it can also be `icmp` (ping traffic), `esp` (VPN traffic), `ospf` (a routing protocol), or any of a number of other protocols.

You might be wondering at the use of "words" instead of "numbers" in the tuple. Well, the "words" are just convenient names for well known IP protocol numbers. You can look up the protocol numbers on the [IANA site](#)

- Source Address: The IP address (v4 or v6) that is the source of the packet
- Source Port: The source port number for the packet. Most commonly this would be for TCP or UDP protocol.

If left off, ANY source port would match the ACE

- Destination Address: The IP address (v4 or v6) that the packet is destined for
- Destination Port: The destination port number for the packet. Most commonly this would be for TCP or UDP protocol

If left off, ANY destination port would match the ACE

Some protocols, such as ICMP, use a "message type" instead of a port number. For example, an ICMP echo is message "Type 8" and an echo-reply is a "Type 0"

By combining these 5 pieces of data, extended access control lists offer flexibility in matching specific types of traffic with each access control entry.

### Creating the ACL

We'll use a named ACL because the self-documentation available with a name is almost always worth the slightly extra typing.

We'll start with the rules for the web site access.

```
ip access-list extended FROM-CLIENTS
  permit tcp host 192.168.0.17 host 172.16.0.65 eq 8080
  deny tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq 8080
  permit tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq 80
```

Make note of how the `permit` for the admin server is listed before the `deny`. This is because of the need to put more specific entries before more general ones. If they were reversed, the `permit` would never be matched.

Before we add the explicit permit at the end, take a look at the current state of the ACL.

```
router# show ip access-list FROM-CLIENTS

Extended IP access list FROM-CLIENTS
 10 permit tcp host 192.168.0.17 host 172.16.0.65 eq 8080
 20 deny tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq 8080
 30 permit tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq www
```

First up, see how port 80 is listed as `www` in line 30? That is because many of the well known application/port details have aliases in IOS (and other networking operating systems). When available, you can leverage the "friendly name" instead of the port.

And second, the default line numbers assigned to ACLs start at 10 and increment by 10s. You can override the line number and choose your own to insert ACEs into an ACL, or to purposefully space them out. Like we will in the next step.

Let's add the explicit permit as line 1000 to provide plenty of room for additional ACEs in the future.

```
ip access-list extended FROM-CLIENTS
 1000 permit ip any any
```

This statement will allow any IP based traffic (which is nearly all traffic) from anywhere to anywhere to pass through this ACL and interface unaffected.

Now we're ready to apply the interface

## Applying the ACL

With the ACL created, we simply need to apply it to the interface on `router`.

```
interface e0/1
 ip access-group FROM-CLIENTS in
```

And confirm it is applied to the interface with the `show ip interface` command.

```
router# show ip interface e0/1 | inc access list

Outgoing Common access list is not set
Outgoing access list is not set
Inbound Common access list is not set
Inbound access list is FROM-CLIENTS
```

If you're like me, you're wondering about the `Common access list` in the output. This refers to an advanced feature called ACL Chaining which is out of scope for CCNA.

## Testing the ACL

Finally, verify that the ACL is operating as expected by running these tests.

- From `client1`
  - Access to `http://server` should work
  - Access to `http://adminserver:8080` should work
  - Access to `http://internet100` should work
  - Pinging `internet100` should work
  - Pinging `server` should work
- From `client2`
  - Access to `http://server` should work
  - Access to `http://adminserver:8080` should NOT work
  - Access to `http://internet100` should work
  - Pinging `internet100` should work
  - Pinging `server` should work

After all the tests are complete, check the ACL again with the `show` command on `router`. You should see matches on all ACEs.

```
router# show ip access-list interface e0/1

Extended IP access list FROM-CLIENTS
 10 permit tcp host 192.168.0.17 host 172.16.0.65 eq 8080 (10 matches)
 20 deny tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq 8080 (1 match)
 30 permit tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq www (16 matches)
1000 permit ip any any (62 matches)
```

## Lab 2: Allow Only Client 1 to Ping Simulated Internet Hosts

In our second lab we have been asked by security to prevent all clients other than `client1` (using its primary IP address) from sending ping packets (ie `echo`) to *Simulated* Internet Hosts. Clients should still be able to ping hosts on private IP ranges as detailed in RFC1918.

Note: When attempting to match "Internet destinations", use the default network as the destination - or `0.0.0.0/0`. The keyword `any` can be used in ACEs for this address.

Let's breakdown this policy into some specific details that we can write ACEs for.

- We'll be restricting access from the Client network, so we'll want to add statements to the existing `FROM-CLIENTS` ACL

- We'll need to match on the `icmp` protocol for each of the ACEs
- We need to **ALLOW** `echo` from `client1` to any
- We need to **DENY** `echo` from other clients to any
- We need to **ALLOW** `echo` from all clients to the private IP networks of `192.168.0.0/16`, `172.16.0.0/20`, `10.0.0.0/8`
- The order of the ACEs will be important - be sure the most specific ACEs come before the general ones.

## Creating the ACL

See if you can come up with the correct ACEs to add to the `FROM-CLIENTS` ACL and then compare with the answer.

### ▼ Answer

This configuration will add the new ACEs after the web browsing entries but before the `permit any any` at the end.

```
ip access-list extended FROM-CLIENTS
 100 permit icmp 192.168.0.0 0.0.0.255 192.168.0.0 0.0.0.255 echo
 110 permit icmp 192.168.0.0 0.0.0.255 172.16.0.0 0.0.240.255 echo
 120 permit icmp 192.168.0.0 0.0.0.255 10.0.0.0 0.255.255.255 echo
 130 permit icmp host 192.168.0.17 any echo
 140 deny icmp 192.168.0.0 0.0.0.255 any echo
```

**Note:** The order of the lines 100 - 130 is not critical. As they are all permit statements, and there is no specific/general issue they could be in any order and have the same result. Line 140 *must* come last in this list or it would match traffic intended to hit the previous entries in this list.

Check the complete ACL after changes.

```
router# show ip access-list FROM-CLIENTS

Extended IP access list FROM-CLIENTS
 10 permit tcp host 192.168.0.17 host 172.16.0.65 eq 8080 (10 matches)
 20 deny tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq 8080 (1 match)
 30 permit tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq www (16 matches)
100 permit icmp 192.168.0.0 0.0.0.255 192.168.0.0 0.0.0.255 echo
110 permit icmp 192.168.0.0 0.0.0.255 172.16.0.0 0.0.240.255 echo
120 permit icmp 192.168.0.0 0.0.0.255 10.0.0.0 0.255.255.255 echo
130 permit icmp host 192.168.0.17 any echo
140 deny icmp 192.168.0.0 0.0.0.255 any echo
1000 permit ip any any (780 matches)
```

As this ACL is already applied to the interface, there is nothing we need to do to activate these changes.

Keep in mind updates to an inplace ACL take effect as soon as you configure them. In a production network, it is possible to disrupt traffic or potentially disconnect remote management sessions to a router if ACL changes are not properly planned out.

## Testing the policy is working

We will run the same tests from Lab 1, however the results for some of the tests will change from before.

**Note:** We are re-testing the web site access to ensure our changes have not disrupted other traffic. This is a good practice to get into.

- From `client1`
  - Access to `http://server` should work
  - Access to `http://adminserver:8080` should work
  - Access to `http://internet100` should work
  - Pinging `internet100` should work
  - Pinging `server` should work
- From `client2`
  - Access to `http://server` should work
  - Access to `http://adminserver:8080` should NOT work
  - Access to `http://internet100` should work
  - Pinging `internet100` should NOT work
  - Pinging `server` should work

And check the counters on the ACLs to see which statements matched traffic and which did not.

```
router# show ip access-list FROM-CLIENTS

Extended IP access list FROM-CLIENTS
 10 permit tcp host 192.168.0.17 host 172.16.0.65 eq 8080 (20 matches)
 20 deny tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq 8080 (2 matches)
 30 permit tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq www (32 matches)
100 permit icmp 192.168.0.0 0.0.0.255 192.168.0.0 0.0.0.255 echo
110 permit icmp 192.168.0.0 0.0.0.255 172.16.0.0 0.0.240.255 echo (6 matches)
120 permit icmp 192.168.0.0 0.0.0.255 10.0.0.0 0.255.255.255 echo
130 permit icmp host 192.168.0.17 any echo (3 matches)
140 deny icmp 192.168.0.0 0.0.0.255 any echo (3 matches)
1000 permit ip any any (1110 matches)
```

Does the output match (pun totally intended) your expectations?

## Lab 3: Generate Syslog Messages for All HTTP Traffic to Internet210

In this lab, we will handle a request from the security team to gather details on all web traffic from the inside network to the website `http://internet210`. It seems there is concern that this web provider might be malicious and the cybersecurity team wants to understand what systems and users are using the service. There is no interest in blocking or preventing access to the server at this time, simply to monitor. They have asked to generate syslog messages that can be analyzed later.

This is an interesting use case that does indeed come up in production networks fairly regularly, the desire to "log" hits to a specific ACE in an access list. Thankfully there is a very easy way to enable option the to fulfill this requirement.

But first, where to apply this new policy? Normally an extended ACL would be applied close to the source, but there is no specific source for this policy. We could apply the policy/ACL on ALL interfaces on `router`, but that seems inefficient and prone to error. In this case, applying it outbound on the interface connected to the destination is the most efficient option.

So let's plan what we need to do for this ACL.

- Create a new extended ACL that will be applied outbound on `eth0/2`
- We need to `permit` and `log` web traffic to `internet210`
- We will need to also `permit any` at the end of the ACL to avoid blocking all traffic with the implicit deny at the end

### Creating the ACL

Here is the fairly simple ACL we will need to create. It doesn't actually block/deny any traffic. But not all ACLs will, sometimes we want to simply "match" traffic to take some action on it.

```
ip access-list extended TO-SIMULATED-INTERNET
  permit tcp any host 192.0.2.210 eq www log
  permit ip any any
```

Note: There is also a `log-input` variation that you could try and compare the results.

Check the ACL configuration to make sure it matches what you expect.

```
show ip access-list TO-SIMULATED-INTERNET
```

### Applying the ACL

Go ahead and apply this ACL outbound on the interface

```
interface eth0/2
  ip access-group TO-SIMULATED-INTERNET out
```

### Testing the ACL

From both clients navigate to `http://internet210`. You should have no problems accessing the site.

Now return to the console of `router` and lets check the results.

First, we'll look at the counters like we have before.

```
show ip access-list TO-SIMULATED-INTERNET

Extended IP access list TO-SIMULATED-INTERNET
 10 permit tcp any host 192.0.2.210 eq www log (38 matches)
 20 permit ip any any (41 matches)
```

You should see matches on the ACE for `internet210`. Matches on the `ip any any` will be due to background traffic on the network and are normal.

Now, let's check for the log messages. We don't have a syslog server setup in this lab, so we will check the log buffer.

```
show logging last 5
```

Tip: The log buffer on a network device can be quite long. Use `last #` to limit the output to the most recent number of messages.

Scroll past the logging output heading section and you should see messages that look like these.

```
.Nov 18 02:14:47.898: %SEC-6-IPACCESSLOGP: list TO-SIMULATED-INTERNET permitted tcp 192.168.0.17(58020) -> 192.0.2.
.Nov 18 02:15:44.009: %SEC-6-IPACCESSLOGP: list TO-SIMULATED-INTERNET permitted tcp 192.168.0.33(38434) -> 192.0.2.
.Nov 18 02:20:05.284: %SEC-6-IPACCESSLOGP: list TO-SIMULATED-INTERNET permitted tcp 192.168.0.17(58020) -> 192.0.2.
```

These messages are generated by hits to the ACL. Notice that some of the messages might be for several packets. The router does this to avoid overloading the CPU and impacting performance.

What severity level was this message sent at?

.

.

.

If you said "Informational - 6" you were correct 🌟.

The router was configured to only show "Notification - 5" messages or higher (lower number value) at the console to cut down on noise from other "informational" messages. Otherwise we would have seen these syslog messages about the ACL hits without needing to look at the log buffer.

## Lab 4: Creative Wildcard Matching

### Technical Tagent... The "Wild" in Wildcard Masks

The vast majority of security policies or traffic matching asks that come to network engineers involve matching a specific host, a specific network, or a range of networks. Each of these can be done with fairly straightforward wildcard masks that might look like 0.0.0.255, 0.0.127.255, or 0.3.255.255..

However, wildcard masks could also look like 0.255.0.0, 0.0.64.0, or even something truly *wild* like 3.148.4.34. That's because like subnet masks, a wildcard mask is actually a binary representation of 0s and 1s that are used to identify which bits in a network address must match the address in the ACE for it to trigger.

If every bit in the network address being tested matches the ACE address bit where there is a 0 in the wildcard mask, then it is a match.

Let's see an example of this in action.

Consider this standard ACL.

```
access-list 1 permit 192.168.34.0 0.0.0.255
```

I know... it's not very "wild", we'll get to that in a minute...

Would the address 192.168.34.101 match? By now you probably already know, "yes, of course it will", but let's do the math to see exactly why.

1. First convert the ACL address and wildcard mask to binary

```
192.168.34.0      11000000.10101000.00100010.00000000
0.0.0.255         00000000.00000000.00000000.11111111
```

2. Now convert the address to test to binary

```
192.168.34.101    11000000.10101000.00100010.01100101
```

3. Now compare the bits in the two addresses where the wildcard bits are 0  
(M = Match, x = no match, - = Not tested)

```
Wildcard Mask      00000000.00000000.00000000.11111111
ACL Address         11000000.10101000.00100010.00000000
Test Address        11000000.10101000.00100010.01100101
Result              MMMMMMMM.MMMMMMMM.MMMMMMMM.-----
```

4. All \*significant\* bits in the ACL address and test address match. So this address would trigger this ACL line.

With the theory and math understood, let's look at a more interesting case.

Consider this standard ACL.

```
access-list 2 permit 192.168.34.0 3.148.4.34
```

Which of these addresses would match this ACL?

- 192.168.34.101
- 192.234.15.194
- 194.60.34.34

Not so easy this time right? Let's do the math!

1. First convert the ACL address and wildcard mask to binary

```
192.168.34.0      11000000.10101000.00100010.00000000
3.148.4.34        00000011.10010100.00000100.00100010
```

2. Now convert the addresses to test to binary

```
192.168.34.101    11000000.10101000.00100010.01100101
192.234.15.194     11000000.11101010.00001111.11000010
194.60.34.34       11000010.00111100.00100010.00100010
```

3. Now compare the bits in each address where the wildcard bits are 0  
(M = Match, x = no match, - = Not tested)

192.168.34.101

```
Wildcard Mask      00000011.10010100.00000100.00100010
ACL Address         11000000.10101000.00100010.00000000
```

|                |                                     |          |
|----------------|-------------------------------------|----------|
| 192.168.34.101 | 11000000.10101000.00100010.01100101 |          |
| Result         | MMMMMM--.-MM-M-MM.MMMMM-MM.Mx-MMx-x | NO-MATCH |
|                |                                     |          |
| 192.234.15.194 |                                     |          |
| Wildcard Mask  | 00000011.10010100.00000100.00100010 |          |
| ACL Address    | 11000000.10101000.00100010.00000000 |          |
| 192.234.15.194 | 11000000.11101010.00001111.11000010 |          |
| Result         | MMMMMM--.-xM-M-xM.MMxMx-Mx.xx-MMM-M | NO-MATCH |
|                |                                     |          |
| 194.60.34.34   |                                     |          |
| Wildcard Mask  | 00000011.10010100.00000100.00100010 |          |
| ACL Address    | 11000000.10101000.00100010.00000000 |          |
| 194.60.34.34   | 11000010.00111100.00100010.00100010 |          |
| Result         | MMMMMM--.-MM-M-MM.MMMMM-MM.MM-MMM-M | MATCH    |

So address only 194.60.34.34 would match this example ACL.... not something easy to see without doing the actual comparison. And doing it by hand is slow and error prone. Thankfully the hardware and software in network equipment can do this test very fast.

The reality is that there is rarely a need for an ACL using a wildcard mask as bizare as 3.148.4.34. However, there are times when a policy might involve "match the third octet", "match the fourth subnet of the network", or "match all odd addresses". And in order to efficiently create ACLs for these requirements require an understanding of wildcard matching.

This lab exercise will tackle just such as policy!

## Preventing Even IP Addresses from Accessing the Services

You've been asked to update the extended ACL on the router interface connected to the client network to prevent any even IP address from reaching the Services network. You weren't given a reason for this strange request, just told to make it happen.

First, we can verify that even IP addresses CAN access the server. This is useful to show that our policy changes actually had the effect we expected. Both `client1` and `client2` are configured with a secondary IP address that we can source pings from for this test.

From `client1`

```
ping -c 3 -I 192.168.0.134 server
```

From `client2`

```
ping -c 3 -I 192.168.0.192 server
```

## What makes an address "even"?

If we consider the requirements for this policy

- Add to the existing ACL already on the interface
- Prevent even addresses from reaching the Services network - so `deny ip`
- We'll need a wildcard mask for even addresses

How do we identify "even addresses" in a mask? Well let's look at a few "addresses" in binary and see what makes an "even" address vs odd

|                |                                     |
|----------------|-------------------------------------|
| Even addresses |                                     |
| 0.0.0.2        | 00000000.00000000.00000000.00000010 |
| 192.168.34.10  | 11000000.10101000.00100010.00001010 |
| 10.11.34.130   | 00001010.00001011.00100010.10000010 |

|               |                                     |
|---------------|-------------------------------------|
| Odd addresses |                                     |
| 0.0.0.7       | 00000000.00000000.00000000.00000111 |
| 192.168.34.11 | 11000000.10101000.00100010.00001011 |
| 10.11.34.129  | 00001010.00001011.00100010.10000001 |

The difference between an "even" and "odd" address is the final bit. Which if you consider the decimal value of the bit positions, that should make sense.

|               |     |    |    |    |   |   |   |   |
|---------------|-----|----|----|----|---|---|---|---|
| Bit Position  | 8   | 7  | 6  | 5  | 4 | 3 | 2 | 1 |
| Decimal Value | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

Any binary value where the final bit is 1 will be odd. Binary values with a 0 in the final bit will be even.

So we need a wildcard mask, and associated address that will ensure that the last bit in an address is a 0.

|               |                                     |
|---------------|-------------------------------------|
| In Binary     |                                     |
| Wildcard Mask | 11111111.11111111.11111111.11111110 |
| Address       | -----.------.------.------0         |

|               |                 |
|---------------|-----------------|
| In Decimal    |                 |
| Wildcard Mask | 255.255.255.254 |
| Address       | 0.0.0.0         |

The only bit in the address that matters is the final one, so it could be any "even" IP address, but it is typical to use 0 for bits that don't matter.

## Updating the ACL

A lot of planning and exploration to get here, but the time has come to add our new ACE to the ACL.

```
router# show ip access-list FROM-CLIENTS
```

```
Extended IP access list FROM-CLIENTS
 10 permit tcp host 192.168.0.17 host 172.16.0.65 eq 8080 (20 matches)
 20 deny tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq 8080 (2 matches)
 30 permit tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq www (32 matches)
100 permit icmp 192.168.0.0 0.0.0.255 192.168.0.0 0.0.0.255 echo
110 permit icmp 192.168.0.0 0.0.0.255 172.16.0.0 0.0.240.255 echo (13 matches)
120 permit icmp 192.168.0.0 0.0.0.255 10.0.0.0 0.255.255.255 echo
130 permit icmp host 192.168.0.17 any echo (3 matches)
140 deny icmp 192.168.0.0 0.0.0.255 any echo (3 matches)
1000 permit ip any any (30211 matches)
```

But where should we put it in the list? The policy is to prevent **ALL** access to the Services network from even addresses, so we want to make sure traffic isn't mistakenly allowed by an earlier line in the ACL. So to be safe, we'll put it up at the top.

```
ip access-list extended FROM-CLIENTS
 1 deny ip 0.0.0.0 255.255.255.254 172.16.0.0 0.0.0.255
```

It looks a little strange, but that doesn't matter if it works. Let's find out!

## Testing the ACL

We'll use the same pings tests from the clients as before to check our ACL. Both of these should show 100% packet loss.

From client1

```
ping -c 3 -I 192.168.0.134 server
```

From client2

```
ping -c 3 -I 192.168.0.192 server
```

Probably good to make sure both clients can still ping the server from their primary IP addresses.

```
ping server
```

Check the counters on the ACL to see if the new ACE matched as expected.

```
router# show ip access-list FROM-CLIENTS

Extended IP access list FROM-CLIENTS
 1 deny ip 0.0.0.0 255.255.255.254 172.16.0.0 0.0.0.255 (6 matches)
 10 permit tcp host 192.168.0.17 host 172.16.0.65 eq 8080 (20 matches)
 20 deny tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq 8080 (2 matches)
 30 permit tcp 192.168.0.0 0.0.0.255 host 172.16.0.65 eq www (32 matches)
100 permit icmp 192.168.0.0 0.0.0.255 192.168.0.0 0.0.0.255 echo
110 permit icmp 192.168.0.0 0.0.0.255 172.16.0.0 0.0.240.255 echo (14 matches)
120 permit icmp 192.168.0.0 0.0.0.255 10.0.0.0 0.255.255.255 echo
130 permit icmp host 192.168.0.17 any echo (3 matches)
140 deny icmp 192.168.0.0 0.0.0.255 any echo (3 matches)
1000 permit ip any any (30367 matches)
```

Before finishing the testing, feel free to check that the access from the clients to the different websites still aligns to the policies from previous labs.

## Lab 5: Uses for ACLs Other Than Interface Filtering

In our final lab, we'll take a look at how ACLs are often part of securing control plane elements of routers.

### Device Access

While username and passwords are used to ensure only authorized users are allowed to log into our routers and switches, it is good practice to limit the source networks or hosts from where network administrators initiate their administrative sessions. This prevents the risk of denial of service attacks, or attempts to brute force access.

By attaching an ACL, most often a standard access list, to the VTY lines you can enhance the security of your network devices.

So let's create an ACL to make sure that only `client1` can be used to administer our router.

```
ip access-list standard ALLOWED-DEVICE-ADMIN
 permit host 192.168.0.17
```

To attach it to the VTY lines, use the `access-class` statement.

```
line vty 0 4
 access-class ALLOWED-DEVICE-ADMIN in
```

Now verify that SSH access to the router only works now from `client1`.

```
ssh cisco@192.168.0.1
```



## SNMP Polling and Trapping

Like SSH, SNMP is a service that benefits from additional security protection than just a community string or credentials for SNMPv3. You can attach an ACL to the `snmp-server` configuration to ensure only authorized sources of traffic are allowed to poll the router.

First, look at the initial SNMP configuration on the router.

```
router# show run | sec snmp

snmp-server community CCNAPREP RO
```

Let's create and add an ACL that will only allow `client2` to sent SNMP requests to the router.

```
ip access-list standard ALLOWED-SNMP
 permit host 192.168.0.33
```

And update the SNMP server config to add this ACL.

```
snmp-server community CCNAPREP RO ALLOWED-SNMP
```

And verify that only `client2` can successfully poll the router with the community string.

```
snmpwalk -v 2c -c CCNAPREP 192.168.0.1 1.3.6.1.2.1.1.5.0
```

## NTP Synchronization

Many network and systems administrators will protect their NTP servers from overload by controlling what hosts are allowed to synchronize with specific masters. This can be done with ACLs as well. Aren't they powerful and flexible! (ACLs, not network and systems administrators...)

We'll update the NTP configuration on `router` to only allow `client1` to communicate with it.

First, let's see the starting configuration.

```
router# show run | sec ntp

ntp master 5
ntp server 10.1.1.1
```

And create the ACL

```
ip access-list standard ALLOWED-NTP
 permit host 192.168.0.17
```

And apply the ACL with the `ntp access-group` command.

```
ntp access-group serve ALLOWED-NTP
```

Note: There are several different options for the types of access-group policy choices in addition to `serve` as used in this example. The details of these differences is beyond the scope of this lab and the CCNA exam.

And finally, attempt to sync the time on the client workstations with `router`. Only `client1` should be successful.

```
ntpd -dw -p 192.168.0.1
```

## NAT Matching

The last service we'll look at in this lab is how ACLs can be used in NAT to identify the traffic that should be translated. In fact, this is something we've already covered in CCNA Prep back in [Season 2, Episode 3 Navigating NAT: Bridging Private Networks to the Internet](#). Since we've covered it in detail in another lab, we'll just look at the NAT and ACL configuration on `router` that is already in place to allow the `client` hosts to reach the public internet during setup to install the SNMP tools used in this lab.

Display the relevant parts of the configuration on `router` with this command.

```
router# show run | sec SETUP-NAT

ip nat inside source list SETUP-NAT interface Ethernet0/3 overload

ip access-list standard SETUP-NAT
 10 permit 192.168.0.0 0.0.0.255
```

The `ip nat inside` statement uses a `source list` to identify traffic to be translated. And that `list` is a standard ACL called `SETUP-NAT`. That ACL will match any traffic from host on the full client network `192.168.0.0/24`.

But after this lab experience, it should come as no surprise that you could configure the ACL so that only some of the clients, and even some types of traffic from some clients, were NAT'd based on a variety of options available in ACLs. While a standard ACL is used in this basic example, an extended ACL could be used as well.

Consider how you might limit NAT to outbound http or https traffic from the clients to public Internet sites. Or how you could prevent NAT to specific destinations?

## Great Job!

Excellent work once again! This lab was designed to put you in the control seat of the traffic entering, leaving, and managing your router through the power of Access Control Lists. And while we focused much of the lab on the flexibility of extended ACLs, in Lab 5 we saw that standard ACLs still have their power and usefulness.

If you'll permit me to say so, you can't deny that you are a match for hands on CCNA studies!

Bonus Content: ACLs are also used in NAT configurations. To see how, check out ***CCNA Prep Season 2 Episode 3 Navigating NAT: Bridging Private Networks to the Internet***